

Checkbox Detection

Solution overview

The README has the commands to run the application and the description of every setting.

1. Summary

The application receives an image (PNG or JPEG) or a PDF and returns where the checkboxes are and whether each one is checked. It has two services: a Java (Spring Boot) backend and a React web interface.

Approach. Classic image processing in plain Java, with no machine learning: the page is converted to black and white, the vertical sides of checkboxes are located, candidate squares are built, rules discard what is not a checkbox, and the ink inside each box decides whether it is checked.

Result. On the three sample PDFs: precision 100 % and recall 94.8 % (section 4).

Contract. Two endpoints receive the file as multipart form data and share the same detection:

```
POST /detect      { "boxes": [ { "bbox": [x1, y1, x2, y2], "is_checked": true }, ... ] }
POST /v2/detect  { "boxes": [ { "page": 1, "bbox": [x1, y1, x2, y2], "is_checked": true }, ... ],
                  "pages": [ { "page": 1, "width": 2550, "height": 4200 }, ... ] }
```

`bbox` is the top-left and bottom-right corner of the checkbox, in pixels of its page (origin at the top left; `x2` and `y2` are the first pixel after the checkbox, so the width is `x2 - x1`). A PDF page is the image drawn at 300 dpi. Errors use the standard *problem+json* format.

Two endpoints: the challenge contract and an extended one

Aspect	/detect (challenge contract)	/v2/detect (extended)
Response	Only the <code>boxes</code> list.	<code>boxes</code> plus the <code>pages</code> list (size of each page).
Each checkbox	Only <code>bbox</code> and <code>is_checked</code> .	Adds <code>page</code> (page number).
PDF	The boxes of all pages, listed one page after another in page order.	The same boxes, each with its page, plus the size of every page.
Used by	Any consumer that follows the challenge.	The web UI.

Why two endpoints. The challenge asks for a precise request and response, and an existing contract has to be followed: other teams depend on the interface, and extra fields can break clients with strict schema validation or code generators. A published contract is versioned, not modified. The extra information that PDFs need (the page of each box and the size of each page, to scale coordinates) therefore lives in a separate, versioned endpoint, and `/detect` keeps the challenge shape exactly.

How one is derived from the other. Both responses are built from the same detection result. The challenge response is obtained by listing the boxes of each page, following the order of the pages, and dropping `page` and `pages`. For a multi-page PDF the page of a box cannot be told apart in that response, so a client that needs it uses `/v2/detect`. The only addition to the challenge input is that `/detect` also accepts PDFs; images behave exactly as specified.

Contract tests. Automated tests assert the exact fields of both responses (images and PDFs): `/detect` has only `boxes`, with `bbox` and `is_checked` in each; `/v2/detect` adds `page` and `pages`. Any extra or missing field fails the build, so a change cannot reach consumers unnoticed.

2. Architecture

2.1 Deployment and frontend

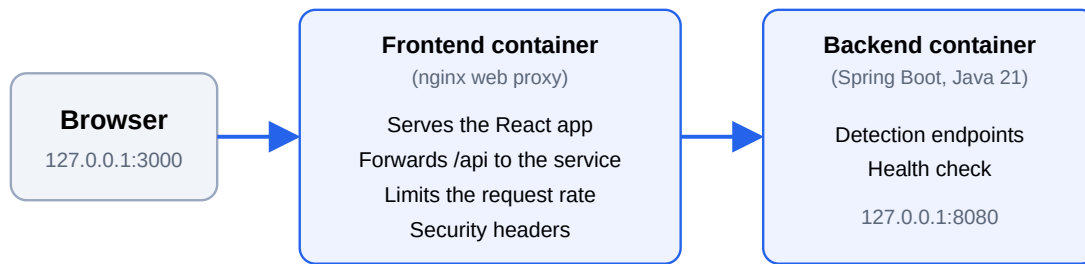


Figure 1. The two deployed services.

Ports are published on `127.0.0.1` only (demo, no authentication). Containers run unprivileged, with a read-only filesystem and memory, CPU and process limits. The React frontend (which calls `/v2/detect`) keeps components that only draw, with state and operations in separate functions; it shows a preview (PDFs are rendered in the browser), draws the checkboxes (green if checked, red if empty) and a summary.

2.2 Backend layers

A Maven project with three modules. Dependencies point toward the models module: the web layer does not know how detection works, and the detector knows nothing about HTTP.

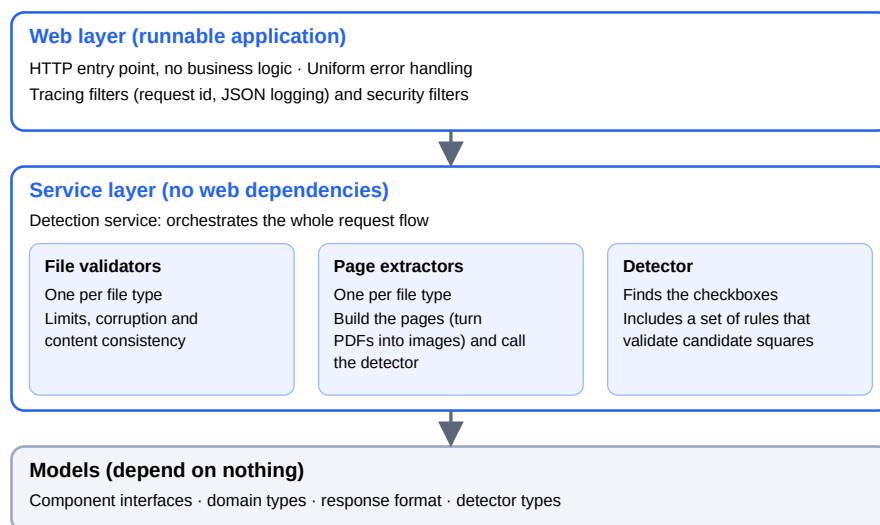


Figure 2. Backend layers. Arrows mean “depends on”.

- **Interfaces in the models module.** Each layer knows only interfaces. Replacing the detector (for example with a machine learning model) means implementing one interface; nothing else changes.
- **One validator and one extractor per file type.** The service indexes them at startup and fails if an enabled type lacks one. Adding a format means adding two classes and enabling it.
- **Independent detection rules.** Each rule is a class. The detector runs them from cheapest to most expensive and stops at the first one that rejects a candidate.
- **Flat web layer.** It receives the file, calls the service and builds the response. One central handler turns any error into a standard response without leaking internal details.

2.3 Request flow

1. Filters assign a request id and add security headers.
2. The real file type comes from the signature of the first bytes (PNG, JPEG or PDF), never from the declared content type.

3. The type-specific validator checks size limits and corruption (for a PDF also the page count and the rendered size of every page).
4. The extractor turns the file into pages (a PDF becomes one 300 dpi image per page); the detector analyzes each page, and one JSON log line is written.

2.4 Security

- **Untrusted input.** Dimensions are read from the header and checked before anything is decoded, which rejects de-compression bombs (a tiny PNG declaring 30000 × 30000 pixels). Password-protected PDFs are rejected.
- **No information leaks.** An unexpected error returns a generic message, and the detail goes only to the log. Only the health check is exposed.
- **Resources.** At most 8 simultaneous requests, 5 requests per second per IP at the proxy, and size limits at proxy and server.
- **Logging and headers.** JSON logs with a request id and never the image content; restrictive content security policy; no CORS, since the UI calls the API from the same origin through the proxy.

3. Image processing

Term	Meaning
Pixel	Each point of the image. Every measure of the detector is in pixels.
Ink and paper	Result of converting to black and white: each pixel is “ink” (dark stroke) or “paper” (background).
Scale	How many times the page is enlarged, in width and height, before analysis. ×1 is the original; ×2 has four times the pixels; ×3, nine; ×4, sixteen.
dpi	Dots per inch: the resolution at which a PDF page is drawn. At 300 dpi a letter page is 2550 × 3300 pixels.

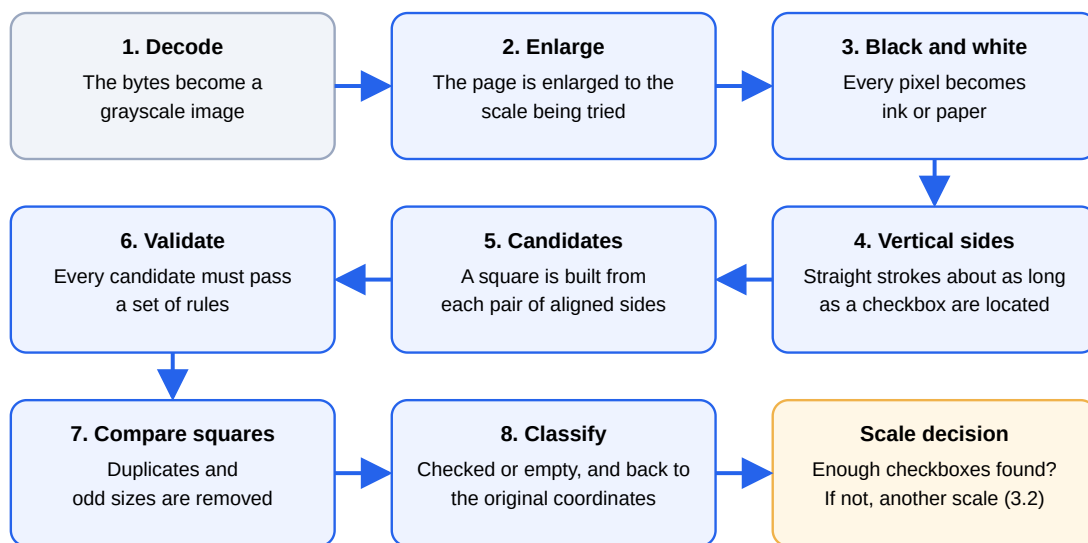


Figure 3. Detector steps. Steps 2 to 8 are repeated for every scale tried.

3.1 Black and white

The image is decoded to grayscale (0 is black, 255 is white) and every pixel becomes ink or paper. Three modes exist:

- **Global.** One gray level for the page, the one that best separates its light and dark pixels (Otsu's method). Sharp on PDFs and scans, but it loses faint lines and is distorted by dark backgrounds such as a PDF viewer in a screenshot.
- **Local.** Each pixel is compared with the average of its surroundings. It keeps faint 1-pixel lines and ignores uniform dark backgrounds, with somewhat more noise.

- **Combined (default).** A pixel is ink if either method says so. It keeps the advantages of both, with a few more false detections on text.

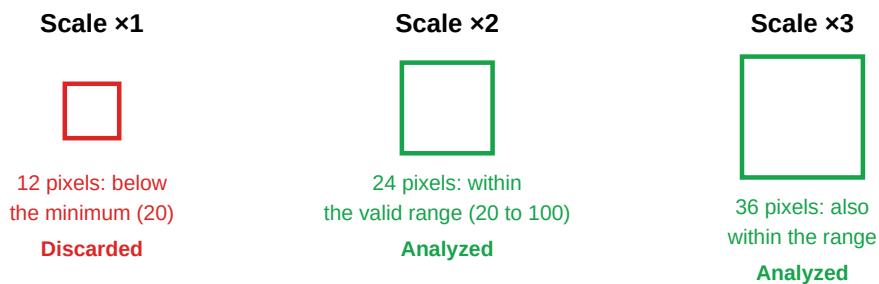
3.2 Scales: what they are and why they change

What a scale is

The factor by which the image is enlarged before looking for checkboxes. At $\times 2$ every original pixel covers 2×2 pixels; at $\times 3$ and $\times 4$, 3×3 and 4×4 . Bicubic interpolation smooths the new pixels instead of making blocks. Scales 1, 2, 3 and 4 are tried, in that order.

Why. The detector rules use absolute pixels: a checkbox is between 20 and 100 pixels wide. The minimum prevents two adjacent letters that form a square from being taken for a checkbox. Those values fit a PDF at 300 dpi, where the checkboxes of the samples measure 30 to 55 pixels. A **screenshot** of the same PDF shows the page much smaller: checkboxes measure 10 to 17 pixels and their border is a faint 1-pixel line. They fall below the minimum and are discarded; before scales were added, no checkbox was found in screenshots.

Solution. Instead of changing the rules, the image is enlarged until the checkboxes reach the size the rules were designed for: a 12-pixel checkbox measures 24 at $\times 2$ and 36 at $\times 3$.



Illustrative drawings: proportions are not to real scale.

Figure 4. The same screenshot checkbox at different scales.

- Scale $\times 1$ is tried first. With 5 or more checkboxes found, the search ends. A PDF page or a scan almost always ends here.
- With fewer than 5, the checkboxes are assumed to be too small and the process repeats at $\times 2$, then $\times 3$ and $\times 4$. It stops at the first scale that reaches 5; if none does, the scale that found the most is returned.
- An enlargement is skipped if the enlarged image would exceed 40 million pixels (cost grows with the square of the factor). Scale $\times 1$ is never skipped.
- Coordinates are divided by the factor, rounding outward, so the response is always in pixels of the original image.

Why not always the largest scale. Each scale costs time and memory in proportion to its square, and enlarging too much can make letters or drawings reach checkbox size and cause false detections. The cheapest scale goes first, and the search stops as soon as the evidence is enough.

Result. Screenshots were simulated by reducing the sample pages with smooth interpolation, as a PDF viewer shows them. The detections amount to 84 % of the real checkboxes at 150 dpi, 85 % at 120 dpi, 64 % at 96 dpi (a typical viewer at 100 % zoom) and 39 % at 72 dpi (about 95 % at the original 300 dpi). It varies by form: at 96 dpi 80 % on the clean page of form 1, but 26 % on its detail page, whose checkboxes are small. These are detection counts against the real total and were not checked box by box. Zooming in before capturing is recommended.

3.3 Sides and candidate squares

The vertical sides are located first, then a square is built from each pair of aligned sides. Whole squares are not searched directly because 23 % to 39 % of the checkboxes share a border with table lines: they do not exist as a separate shape, and only their vertical sides give them away. When two sides are much taller than the distance between them (stacked checkboxes), they are cut at the horizontal lines that cross them. This step is permissive on purpose; the next rules decide.

3.4 Validation rules

A candidate is a checkbox only if it passes all four rules, evaluated from cheapest to most expensive.

Rule	Requires	Discards
1. Size and proportion	Side of 20 to 100 pixels; width/height between 0.8 and 1.25.	Letters, small strokes, elongated rectangles.
2. Ink in the corners	All four corners contain ink.	Round letters ("o", "O", "Q"), whose corners are empty.
3. Free side	At least one side without ink right next to it.	Strokes from the middle of a word, with letters on both sides.
4. Closed outline	All four sides drawn, with no gaps.	Pairs of letter strokes ("H", "ng").

3.5 Comparison between squares and classification

- **Duplicates.** If two candidates overlap by 40 % or more, the smaller one is kept.
- **Dominant size.** All checkboxes of a form have the same size; rare sizes are discarded.
- **Checked or empty.** The ink inside the box, without its border, is measured: above 7 % the box is checked. Any mark counts (X, tick or scribble).

PDF. Each page is drawn at 300 dpi and follows the same process as an image. The detector takes about 0.2 s per 2550 × 4200 page, excluding rendering.

4. Measured quality: precision and recall

Definitions. TP: detected checkbox that is real. FP: detection that is not a checkbox. FN: real checkbox not detected.

Precision = $TP / (TP + FP)$. **Recall** = $TP / (TP + FN)$. **F1**: harmonic mean of both.

Method. The three sample PDFs were evaluated (form 1 with two pages, form 2 and form 3), at 300 dpi.

- **Precision:** a crop of each of the 271 detections was reviewed by hand, checking that it is real and that its state (checked or empty) is right.
- **Recall:** missed checkboxes were looked for with a second, independent method (square outlines found by templates) and by inspecting every page with the detections drawn on it.
- A detection matches a real checkbox when they overlap by at least 0.5 (intersection over union).

Document	Real	TP	FP	FN	Precision	Recall	F1
Form 1, page 1	117	105	0	12	100 %	89.7 %	94.6 %
Form 1, page 2	42	40	0	2	100 %	95.2 %	97.6 %
Form 2	48	48	0	0	100 %	100 %	100 %
Form 3	79	78	0	1	100 %	98.7 %	99.4 %
Total	286	271	0	15	100 %	94.8 %	97.3 %

- **State:** all 271 detections have the right state. There are 92 real checked boxes (89 detected) and 194 empty ones (182 detected).
- **Control without checkboxes:** 4 false detections on 60 text pages (the initial version produced 319).

The 15 missed checkboxes:

Where	Count	Cause
Form 1, page 1, "Neighborhood" tables and nearby	10	They measure 57 × 45 pixels (proportion 1.27); rule 1 accepts up to 1.25.
Form 1, page 1, "Disposal" and "Other"	2	Neighboring strokes fuse with the checkbox: no candidate, or an oversized one.

Form 1 page 2 (2) and form 3 (1)	3	Pen strokes over the corners (rule 2), dotted border (no continuous sides), gap in a faint outline (rule 4).
----------------------------------	---	--

Scope of the measurement

The ground truth was built by one person, with the help of an auxiliary method: a checkbox nobody saw is not counted, and the real recall would be somewhat lower. The detector thresholds were also tuned on these same pages, so the results are optimistic for new forms.

Robustness tests

- **API.** 44 files: unsupported types (text, HTML, SVG, ZIP, executable, GIF, BMP, TIFF, WebP), empty, truncated and garbage images and PDFs, declared dimensions of 30000 × 30000, a decompression bomb, PDFs with 21 pages, no pages, a password and a 200-inch page, and files of 9.5 to 30 MB. None produced a 500; each received the expected 400 or 413 with a clear message. Wrong methods, field names, content types and hostile file names also behaved correctly.
- **Proxy and load.** 413, 429 and 503 (backend stopped) with JSON messages, and automatic recovery. 16 simultaneous 4000 × 4000 images and 8 simultaneous 7000 × 7000 images succeed; PDFs do not (section 5).
- **Frontend.** 54 scenarios in a real browser (invalid and oversized files, corrupt PDFs, server and network failures, drag and drop, keyboard, hostile names, mobile, dark mode). Three defects were fixed: an unexpected response shape blanked the page, a password-protected PDF showed a raw library message, and a PDF without pages gave no message.

5. Decisions and limitations

Decisions

- **Synchronous processing.** The client waits for the response. Simple, but it does not scale with large files or high concurrency (section 6).
- **Two endpoints.** `/detect` follows the challenge contract exactly and `/v2/detect` adds page numbers and page sizes for PDFs; both are built from the same result and covered by contract tests (section 1).
- **No authentication.** Intentional and pending. Current protection: concurrency cap, proxy rate limit (not applied when port 8080 is used directly) and local-only ports.
- **Simple, slightly redundant steps.** A PDF is opened once to validate and once to render, each page goes through a PNG before the detector, and uploads are not chunked (10 MB maximum). All are cheap next to rendering.
- **Tests run inside the image build** (nothing untested ships, but builds are slower); backend image of about 520 MB; only PNG, JPEG and PDF.

Limitations

- **Concurrent PDFs exhaust memory.** A 20-page PDF peaks near 1.7 GB of the 2 GB container, and two or three simultaneous ones exhaust the heap: the JVM exits, the container restarts and the requests in flight fail. The cap of 8 threads does not protect memory for PDFs, although 8 simultaneous 50-million-pixel images are fine.
- **Damaged files that decoders can partly read are accepted.** A truncated JPEG, or a PDF cut in half that the parser repairs, is answered with 200 and few or no checkboxes instead of an error.
- **PDF conversion is the weak point.** The library cannot be interrupted and does not limit decompressed stream sizes: a malicious PDF could consume CPU or memory. Mitigated by size, page and pixel limits, the container memory limit and automatic restart.
- **Screenshots.** Only part of the checkboxes is recovered, about 64 % at 96 dpi and 39 % at 72 dpi in the simulation (section 3.2).
- **False positives on text.** Some letter combinations draw a square, such as an “E” followed by an “l”.
- **Robustness.** Sizes are absolute; scans tilted more than about one degree lose checkboxes; photos are not supported.
- **The application's own frames.** In a screenshot with the colored frames of this UI, the returned box can be the frame.

- **First version of the detector.** No deskewing, and thresholds tuned on the challenge samples.

6. Proposed evolution: asynchronous processing

The detector performs heavy calculations (binarization, up to four scales, candidate search and validation), and a PDF is rendered at 300 dpi before analysis. With synchronous processing, a few heavy requests occupy the server threads (the cap is 8) and degrade the service for everyone, including users who send small files.

Proposal. The user uploads the file to a bucket (S3, R2 or compatible); the end of the upload triggers processing, and the result is communicated asynchronously (email, webhook, notification).

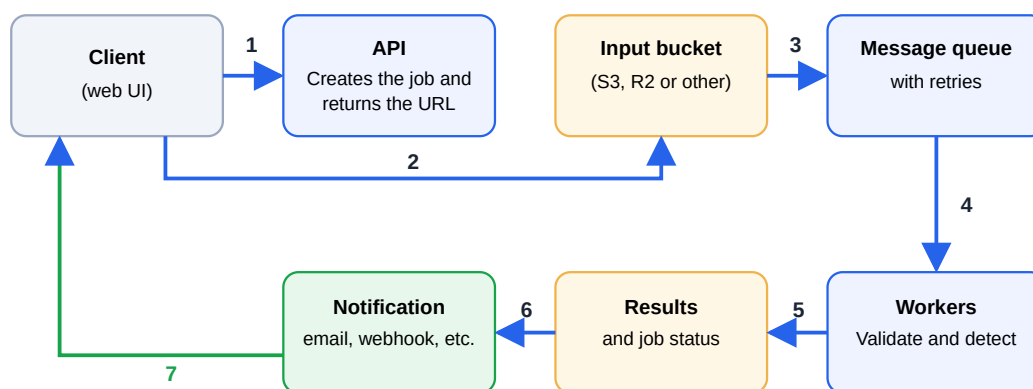


Figure 5. Proposed asynchronous flow. Numbers match the steps below.

1. **Ask for permission.** The API validates the declared type and size, creates a “pending” job and returns a pre-signed upload URL (short expiry, fixed type and maximum size).
2. **Upload directly to the bucket.** The API holds no connections or memory for large files.
3. **Upload-finished event** goes to a message queue. If the bucket cannot emit it, the client confirms the upload and the API enqueues the message.
4. **A worker takes the job** from the queue, validates the file in depth, converts PDFs, tries the scales and runs the detector.
5. **The result and job status** (“done” or “failed”) are stored.
6. **The user is notified** by email, webhook or UI; the client can also poll the job status with its id.

Advantages. A light API that only creates jobs and queries status. Workers scale with queue length, and the queue absorbs peaks. Automatic retries and a dead-letter queue. PDF conversion runs isolated with strict time and memory limits, which solves the weak point of section 5. Larger, chunked and resumable uploads become possible.

Costs. More pieces to operate (bucket, queue, workers, result storage, notifier). The result is no longer immediate, so the UI must show progress. A queue can deliver a message more than once, so processing must be idempotent. The presigned URL needs a short expiry and strict limits, and nothing declared by the client is trusted, so deep validation happens in the worker. Each job must be tied to an authenticated user, with a defined retention period for files and results.

Hybrid variant. The synchronous path stays for small images (below a size threshold) and the asynchronous one is used for PDFs and large files. Validators, extractors and detector are reused unchanged inside the worker.

7. Improvements

High priority

- **Formal contract.** Publish an OpenAPI specification of both endpoints and keep the contract tests as the guard against accidental changes (section 1).
- **Asynchronous processing** with bucket, queue and workers (section 6).

- **Memory safety with concurrent PDFs.** Render and analyze one page at a time and limit how many PDFs are processed at once, or move them to workers (section 6).
- **Accept the 1.27 proportion** in the size rule: recovers 10 of the 15 missed checkboxes. The cost in false positives has to be measured.
- **Authentication and authorization.**

Detection quality

- Separate the checkbox from neighboring strokes, and tolerate damage (gaps, covered corners, dotted borders) with a score instead of an absolute rejection.
- Make tolerances proportional to checkbox size, to improve low-zoom screenshots, and tune the local threshold with more screenshots.
- Deskew scans, support photos, add rules against false positives on text, and tell a thick frame apart from the checkbox it contains.
- Add a machine learning model as a second opinion for doubtful checkboxes, and a confidence score per checkbox.
- Evaluate with more documents and more than one labeler, on a set separate from the one used for tuning, and automate the measurement.

Performance, security and product

- Process the pages of a PDF in parallel, avoid the intermediate PNG, speed up duplicate removal and choose the scale directly.
- Convert PDFs in an isolated process with time and memory limits (the worker of section 6), and reject truncated JPEG files and PDFs that need repair.
- Add metrics (time per stage, checkboxes, errors), move the tests to a separate stage of the image build and use a lighter backend image.
- Automated frontend tests, more formats (multi-page TIFF, WebP) and manual correction of detections.

8. Implementation notes

The source code carries no comments; the rationale behind non-obvious choices is kept here.

- **Strokes and candidates.** Ink runs in neighboring columns are connected even when they only touch diagonally, so slanted or anti-aliased strokes stay whole; union-find joins them without comparing every pair. There is no maximum stroke length (stacked checkboxes form one long stroke), the thickness limit rejects bold letters, and the minimum length is 80 % of the smallest checkbox because stroke ends can fade. When a thick frame is fused to the thin side of the box below, each side is fitted to the column that really has ink in the middle rows; the anti-aliased halo counts as part of the side, otherwise it would be mistaken for a neighbor.
- **Black and white.** On a pure black-and-white page Otsu's method ties at every level; the middle of the tie is used, because the first level would classify every anti-aliased gray as paper and thin strokes would vanish. The local threshold uses running sums, so its cost does not depend on the window size. Transparent PNG pixels are painted white first, otherwise they would become ink.
- **Rules and comparison.** Rules run from cheapest to most expensive and the first rejection stops the evaluation. The closed-outline rule tolerates 2 pixels (scans can be a pixel or two off) and skips the 2 pixels at each end, which the corner rule covers; the free-side rule ignores the top and bottom rows because table lines continue past the box. Duplicate removal keeps the smallest square and runs before the size analysis, so repeated candidates cannot make a wrong size look common; with fewer than 4 checkboxes in the biggest size group there is no evidence to call another size an outlier. In the classifier the border is excluded because it is ink too.
- **Proxy and containers.** The backend name is re-resolved every few seconds, because a recreated container may get another IP. The proxy body limit (11 MB) equals the backend's, so an oversized upload is rejected at once (with a higher limit, uploads between the two took 10 seconds to be rejected), and proxy errors use the same *problem+json* shape. Fontconfig and a font are installed because Java needs them to render PDFs with non-embedded fonts.